

PHASE 6

CREATING WIDGETS AND ADDING TO THE PAGE

Building the reusable Service Portal components that bring each page to life, and placing them correctly.

Employee Raise Issue — ServiceNow Implementation Guide
Standalone Phase Document 6 of 7

Document Version 1.0

About This Document

This document is a standalone, self-contained implementation guide for Phase 6 — Creating Widgets and Adding to the Page — of the Employee Raise Issue project. It can be read and actioned on its own, without needing the full multi-phase master documentation, though it references the wider project for context where useful.

Field	Detail
Application	Employee Raise Issue (built within the Employee Center scope)
Phase	Phase 6 of 7
Primary Owner	Service Portal / Front-End Developer
Related Document	Employee Raise Issue Documentation (full 7-phase project reference)
Instance Referenced	dev218486.service-now.com
Document Version	1.0

Contents

1. Purpose and Scope
2. Where This Phase Fits in the Project
3. Objective
4. Prerequisites
5. Widget: Landing Banner
6. Widget: My Issues List
7. Screenshot Reference
8. Reused Out-of-Box Widgets
9. Adding Widgets to Pages
10. Performance Considerations
11. Widget Architecture Reference
12. Common Issues and Troubleshooting
 - 12.1 Support and Ownership for These Widgets
13. Validating Phase 6 Completion
14. Best Practices
 - 14.1 Frequently Asked Questions
15. Summary and Handoff to Phase 7
 - 15.1 Phase 6 Deliverables Checklist

1. Purpose and Scope

This document is the complete, standalone implementation guide for Phase 6 of the Employee Raise Issue project: building the custom Service Portal widgets that complete the pages created in Phase 5, and placing them correctly using the Portal Designer.

By the end of this phase, the Employee Issue Home page has a welcoming call-to-action, and the My Issues page shows each employee their own submission history — both built as small, focused, reusable widgets rather than one large monolithic page script.

2. Where This Phase Fits in the Project

- Depends on: the pages created in Phase 5, which these widgets are placed onto, and the Employee Issue table from Phase 2, which the My Issues List widget queries.
- Feeds into: Phase 7, where the rendered widgets are checked as part of the end-to-end functional and UI tests.
- Verified by: Phase 7's UI/UX testing (mobile and desktop rendering) and the portal search checks (TC referencing Figures 11–12).

Relationship	Detail
Upstream dependency	Phase 5's pages must exist before a widget can be dragged onto them; Phase 2's table must be queryable before My Issues List can return anything.
Downstream dependency	Phase 7's UI/UX and functional testing directly exercises the two widgets built here.
Shared ownership	Widget code, once built, is reused wherever it is placed — a change here can affect every page carrying the widget, not just one.

3. Objective

Widgets are the reusable building blocks of a Service Portal page. This phase covers the custom widgets built for the portal, alongside the out-of-box widgets reused from the platform, and documents exactly how each was placed onto its page.

4. Prerequisites

- Phase 5 complete: the portal and its pages exist and are reachable.
- Familiarity with the Service Portal Widget Editor (HTML Template, CSS-SCSS, Client Script, Server Script panels).
- Agreement on the visual style (colors, spacing, iconography) each widget should follow, ideally matching the theme selected in Phase 5.

5. Widget: Landing Banner

A lightweight, presentational widget used on the Employee Issue Home page to welcome employees and direct them to Raise an Issue.

Property	Value
Name	Landing Banner (built as "Commercial Widget" in the working instance — see Figure 9)
Option Schema	title (string), subtitle (string), cta_label (string), cta_page (string)
HTML Template	Renders a hero section with the configured title, subtitle, and a button linking to the Raise an Issue page route.
CSS (SCSS)	Defines a padded hero container, responsive typography, and a primary-colored call-to-action button with a hover state, plus staggered fade/slide-in animation on load.
Client Script	Handles the button click, navigating to the raise-issue page route.

6. Widget: My Issues List

An interactive widget used on the My Issues page, giving employees visibility into requests they have submitted, built directly against the Employee Issue table from Phase 2.

Property	Value
Name	My Issues List
Server Script	Queries the Employee Issue table filtered to requester = gs.getUserID(), sorted by sys_created_on descending, and returns number, short_description, category, priority, and state for each record.
HTML Template	Renders a responsive card/table of issues, each showing the issue number, short description, a color-coded priority badge, and current state.
CSS (SCSS)	Defines the card layout, priority badge colors (green/amber/red), and empty-state styling shown when no issues exist yet.
Client Script	Handles loading state, an empty-state message ("You haven't raised any issues yet"), and a refresh action that re-invokes the server script via the widget's update call.

WATCH OUT: The My Issues List widget's server script must filter strictly by the logged-in user's own sys_id. A missing or incorrect filter here is a data exposure risk — Phase 7's security testing (TC-06) exists specifically to catch this class of defect before go-live.

7. Screenshot Reference

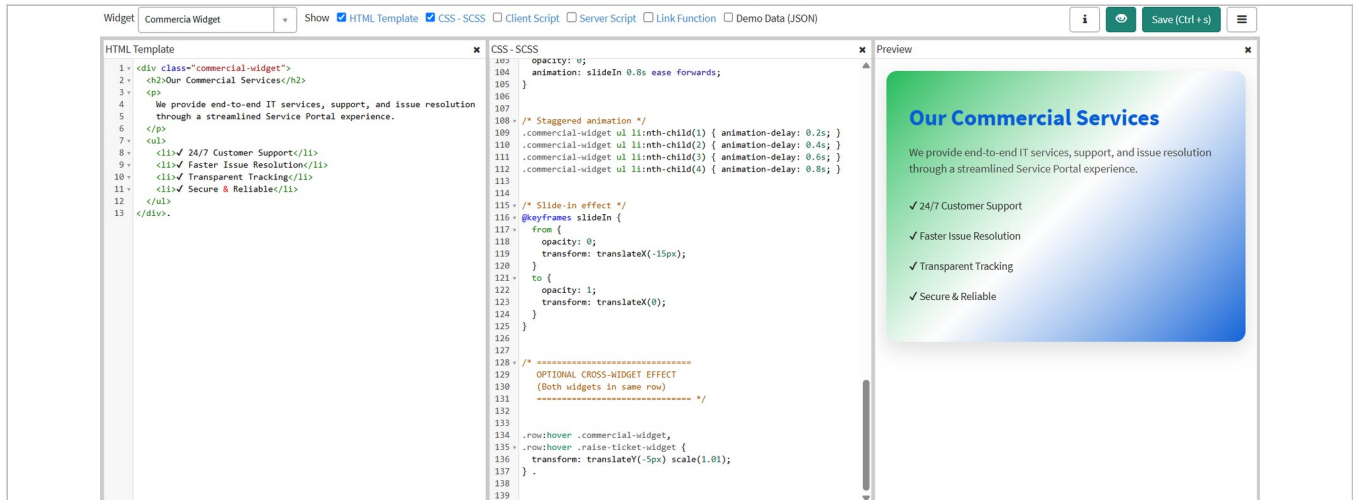


Figure 9 — Service Portal Widget Editor showing the *HTML Template* and *CSS-SCSS* for a supporting widget, with a live preview of the rendered card on the right.

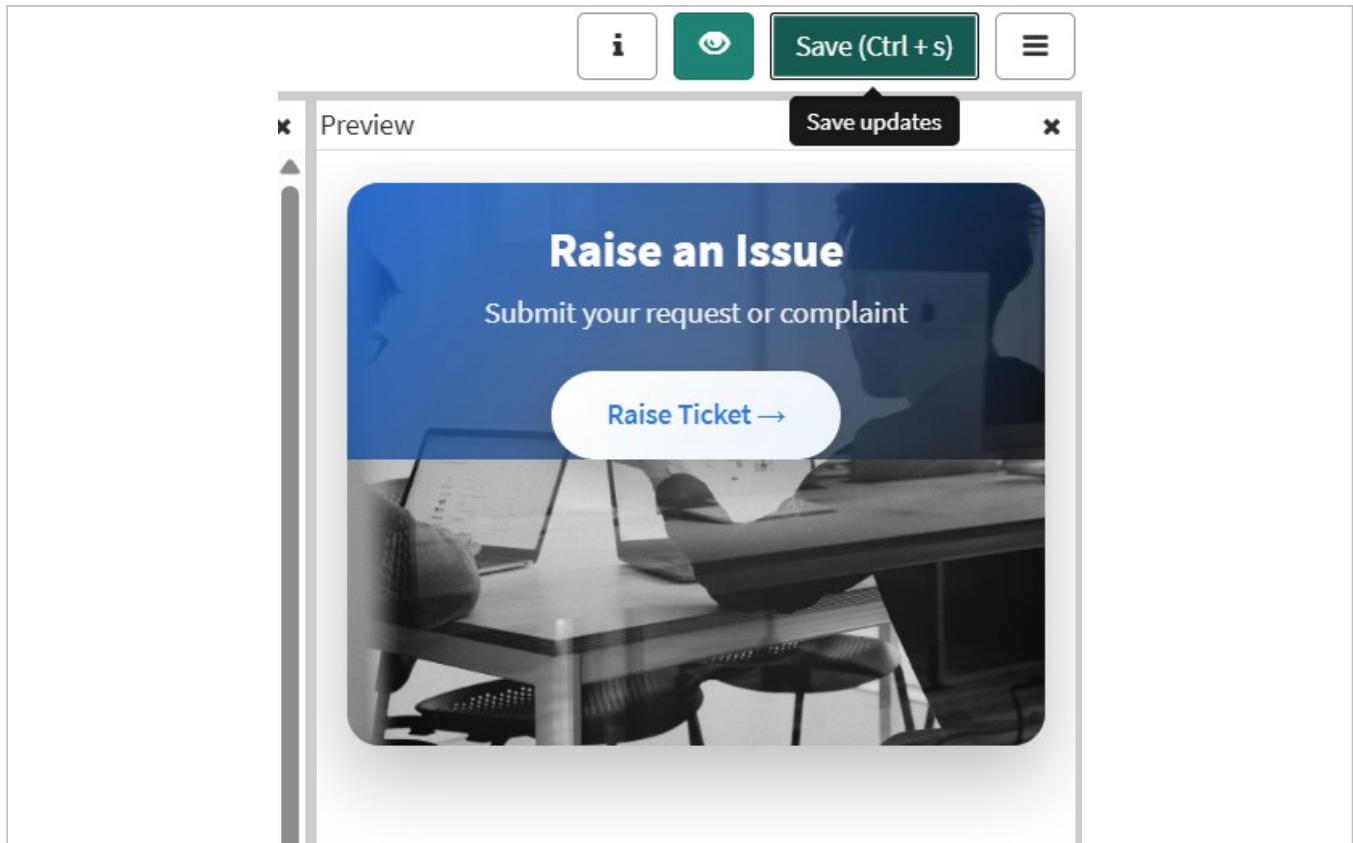


Figure 10 — Rendered "Raise an Issue" widget card as it appears to employees, with a *Raise Ticket* call-to-action linking through to the record producer.

Figure 9 shows the Widget Editor's three-pane layout: code on the left and center, live preview on the right, updated instantly as HTML or CSS is edited. Figure 10 shows the resulting card as an employee would actually see it on the portal, confirming the widget renders correctly end to end.

8. Reused Out-of-Box Widgets

- Record Producer widget — hosts the Raise Employee Issue catalog item on the Raise an Issue page (Phase 4/5).
- Page Title / Breadcrumb widget — provides consistent page headers across all four pages.
- Header and Footer widgets — reused portal-wide from the underlying theme selected in Phase 5.

NOTE: Reusing platform-provided widgets wherever they already do the job (as with the Record Producer widget) is preferable to rebuilding equivalent functionality from scratch — custom widget effort in this phase is reserved for behavior the out-of-box widgets do not provide.

9. Adding Widgets to Pages

1. Open Service Portal > Page in the Portal Designer for the target page (e.g. employee-issue-home).
2. Drag the Landing Banner widget into the primary content row.
3. Open the widget instance options and populate title, subtitle, cta_label, and cta_page.
4. Repeat for the My Issues page, dragging in the My Issues List widget with no additional configuration required.
5. Preview each page as a test employee user (not an administrator) to confirm the experience matches what a real employee will see, including ACL-restricted data.
6. Save the page layout so the widget placement persists.

10. Performance Considerations

- The My Issues List widget's server script limits the query to the most recent 20 records by default, with pagination reserved for a later enhancement if issue volume grows significantly.
- Widget CSS is scoped using the widget's own SCSS block rather than page-level styles, preventing style leakage between widgets sharing the same page.
- Any images used in a banner widget should be served efficiently (appropriately sized, and lazy-loaded where the portal framework supports it) to avoid delaying initial page render.

BEST PRACTICE: Keep server-side widget scripts thin: apply the same encapsulated business logic used elsewhere (Script Includes) rather than duplicating query logic inside the widget itself, so the same logic can be reused by a future REST integration or mobile experience.

11. Widget Architecture Reference

For administrators unfamiliar with the Service Portal widget model, the table below summarizes what each part of a widget is responsible for.

Widget Part	Responsibility
HTML Template	Defines the markup rendered on the page, using AngularJS-style bindings to the widget's data object.
CSS - SCSS	Styles scoped to this widget; compiled and applied only where the widget is placed.
Client Script	Runs in the browser; handles user interaction, local state, and calls to the server script.
Server Script	Runs on the server when the widget loads or is explicitly refreshed; the only place with direct GlideRecord access.

Widget Part	Responsibility
Option Schema	Defines configurable inputs (like title or subtitle) an administrator can set per instance of the widget, without editing code.
Link Function (advanced)	Optional low-level DOM manipulation hook, rarely needed for widgets like the ones built in this phase.

12. Common Issues and Troubleshooting

Issue	Likely Cause	Resolution
Widget preview looks correct in the editor but wrong on the live page	Page-level CSS or another widget's styles are leaking in.	Scope widget CSS more tightly (prefix class names with the widget's own identifier) and re-test.
My Issues list shows other employees' records	The server script's filter on requester is missing, incorrect, or was overridden during testing.	Review the server script's GlideRecord query and confirm it filters strictly to <code>gs.getUserID()</code> .
Widget shows a blank state even when records exist	The client script does not correctly bind to the server script's returned data object.	Confirm the property names used in the HTML Template exactly match those returned by the server script.
Button inside a widget does not navigate anywhere	The Client Script's click handler is missing, or the target page route is misspelled.	Verify the click handler is wired to the button and the page route matches an existing, published page.
Widget works for administrators but not for standard employees	An ACL on the underlying table blocks the employee role from the fields the widget queries.	Test as a genuine non-admin employee account and adjust field-level or table-level ACLs as needed.

12.1 Support and Ownership for These Widgets

Widgets tend to outlive the phase in which they were built, so it is worth naming ownership explicitly rather than leaving it implied:

Area	Owner
Widget HTML, CSS, and Client Script	Service Portal / Front-End Developer
Widget server scripts and their query logic	Platform Administrator, in coordination with the Front-End Developer
Content shown inside a widget (wording, categories)	HR / IT Support, as the business owner of that content
Day-to-day "this widget looks broken" reports	General IT Service Desk, escalating to the Front-End Developer

13. Validating Phase 6 Completion

Check	Expected Result
Landing Banner renders correctly on Employee Issue Home	Title, subtitle, and call-to-action button all display and link correctly

Check	Expected Result
My Issues List renders correctly on My Issues	Shows the logged-in employee's own issues, correctly sorted and labeled
My Issues List filters by user correctly	A second test employee sees only their own records, not the first employee's
Empty state displays correctly	A brand-new employee with no submissions sees a friendly empty-state message, not a blank page or error
Mobile rendering	Both widgets render cleanly on a narrow mobile viewport

14. Best Practices

BEST PRACTICE: Build one widget per distinct piece of functionality rather than one large widget per page — smaller, focused widgets are easier to test, reuse, and hand over to another developer.

BEST PRACTICE: Always test widgets as a genuine non-admin employee account, not as an administrator — administrators often have broader visibility that can mask ACL-related defects a real employee would hit.

BEST PRACTICE: Keep option schemas (title, subtitle, etc.) for presentational widgets so the same widget can be reused on future pages with different wording, without duplicating the widget itself.

14.1 Frequently Asked Questions

Can the same widget be placed on more than one page?

Yes — this is one of the main advantages of the widget model. A single Landing Banner widget, for example, could be reused on a future "IT Announcements" page with different title and subtitle values, without creating a second copy of the code.

Should widget server scripts call other tables directly, or go through a Script Include?

For simple, single-purpose queries like the My Issues List widget's lookup, a direct GlideRecord query in the server script is reasonable. Where the same logic (for example, the category-to-assignment-group routing referenced in Phase 4) is needed in more than one place, move it into a Script Include so both the widget and the Producer script call the same, single source of truth.

How do we know if a widget is a performance risk?

Watch for unfiltered or unindexed queries (a query with no condition on a large table), synchronous calls to external systems inside the server script, and any widget that runs its full query on every page scroll or click rather than caching results client-side where appropriate.

15. Summary and Handoff to Phase 7

At the end of this phase, every page created in Phase 5 is fully functional: Employee Issue Home welcomes employees and directs them to raise an issue, and My Issues gives them closed-loop visibility into what they've already submitted. The complete, end-to-end experience is now ready for structured testing in Phase 7 — Testing and Validation.

15.1 Phase 6 Deliverables Checklist

Deliverable	Status
Landing Banner widget built and configured	Complete
My Issues List widget built and correctly filtered by user	Complete
Both widgets placed on their respective Phase 5 pages	Complete
Out-of-box widgets (Record Producer, header/footer) confirmed reused correctly	Complete
Performance and security considerations reviewed	Complete
Ready for Phase 7 Testing and Validation	Ready